

KBC DIGITAL DIVISION

AdBoard

Digital Commercial Booking & Order System

Build Brief — Version 1.0. The document that tells the engineering team what to build, in what order, and what "done" means.

Product Name	Digital AdBoard
Document Version	1.0 (Build Brief)
Document Type	Engineering specification — for the development team
Product Owner	Chief of Digital Media — KBC
Tech Stack	React JS Firebase Vercel
Methodology	Agile — 2-week sprints, phased delivery
Delivery Strategy	Phase 1: Order Sheet slice (ship fast). Phase 2: Execution & reporting.
Status	Approved for Development

0. Read This First

This is the build document. A separate executive brief explains *why* we are building Digital AdBoard and is for management. **This** document tells you *what* to build and *in what order*. If something is unclear, ask before you build — do not guess, and do not "improve" the workflow rules. The workflow rules *are* the product.

THE ONE RULE THAT MATTERS MOST

The sales executive **configures** a campaign; the **system prices it** and generates the order document. The sales executive **never** hand-types a price and **never** edits the generated Order Sheet. If you find yourself building an editable price field for a salesperson, stop — you have misunderstood the product.

KBC's sales team sells digital advertising using a vague paper "Air-Time Order." That vagueness lets them promise a client one thing, write down another, and leave the Digital team to absorb the gap. We are not fixing the Air-Time Order — that belongs to TV and Radio. We are building a **separate digital system** that produces a **locked, itemised, signed document** (the Digital AdBoard Order Sheet) which removes the ambiguity for anything digital.

1. Problem Statement

KBC's digital advertising is booked on a paper Air-Time Order form that is signed, stamped, and approved by Finance and the Sales Manager. The document exists and the approval chain works. The problem is **what is written inside it**: digital line items are deliberately vague.

This vagueness is a learned commercial behaviour by a sales team whose average age is 50–54, who built their careers selling radio and TV, and who now carry digital targets. An ambiguous order lets a salesperson define deliverables *after* the sale, in their own interest, against no written record. The team calls this "ujanja" (cunning). Real consequences observed in five live orders:

- An order written as "social media activation — 2 hours" with no platforms, quantities, or content defined.
- A boosting budget approved as a handwritten footnote ("approve 5% for boosting") rather than a costed line.
- A **KSh 6,000,000** signed order that grew informally to a **KSh 29,000,000** delivery expectation — a KSh 23M gap with no documentation.
- A 20-day, 64-location national roadshow requiring a travelling digital officer, with the word "digital" absent from the order's instructions.

2. Objective

Build an internal web application — **Digital AdBoard** — that lets the sales team configure a digital campaign, then **generates a locked, itemised, priced Order Sheet** the client and Advertising Manager sign before the Digital team begins work. The app makes scope, pricing, discounts, and boosting explicit and on-record by default.

Digital AdBoard is an internal tool built incrementally, the same way Lineup was. It does not replace the Air-Time Order — it runs alongside it. The only link between them is a hand-written **DAB reference number** the salesperson copies onto the Air-Time Order, exactly as they already hand-write PO references today.

THE SENTENCE THAT DEFINES THE SYSTEM

The Air-Time Order authorises the budget. The Digital AdBoard Order Sheet — generated by the app, signed by the client, countersigned by the Advertising Manager — defines exactly what that budget buys. Without both, the Digital team does not begin.

3. Users & Roles

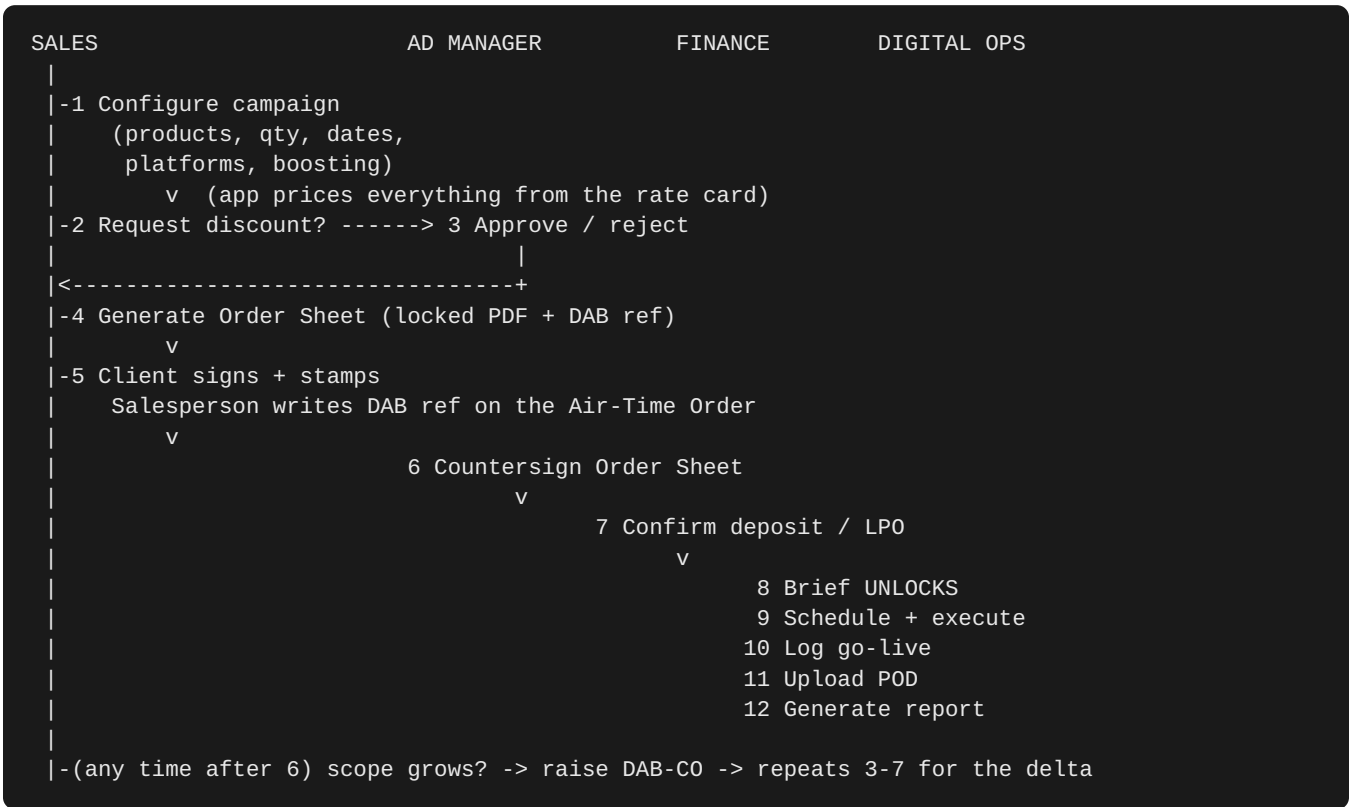
Four roles for Version 1. Permissions are enforced at the **database security-rule level**, not merely hidden in the UI. Roles are managed by the Admin.

Role	Who	Can do	Cannot do
Sales	Sales executives (avg 50–54, broadcast background, low digital fluency)	Configure campaigns, generate Order Sheets, request discounts, raise Change Orders, view own pipeline	Edit prices, edit a generated Order Sheet, approve own discount, mark campaigns delivered, see other reps' deals
Advertising Manager	The commercial gatekeeper	Approve/reject discounts, countersign Order Sheets, view all pipeline	Edit the rate card, execute campaigns
Digital Ops	The Digital Media execution team	Receive unlocked briefs, validate materials, schedule, log go-live, upload POD, generate reports	Create orders, change pricing, approve discounts
Management / Admin	CDM, Head of Commercial, MD (read); Admin also edits rate card	View everything; Admin edits rate card & users	—

DESIGN NOTE — THE SALES PERSONA

These are not young digital natives. Use large tap targets, plain language, no jargon, and can't-get-it-wrong defaults. If an intern who has never seen the app cannot complete a booking in under 10 minutes on a phone, the UX has failed.

4. The Workflow (the spine of the app)



THE HARD GATE

Stage 8 (brief unlock for Digital Ops) is blocked until **all three** are true: signed Order Sheet uploaded, Air-Time Order serial entered, and payment confirmed. There is no override button for Sales.

5. User Stories — Version 1

These are the stories the team delivers for V1. Nothing outside this list is in V1 scope. **Story IDs are stable — use them in commits and PRs** (e.g. `git commit -m "DAB-12: live cost calculator"`).

Epic A — Foundation

ID	As a...	I want to...	So that...
DAB-01	User	log in and be given the right role	I see only what my job needs
DAB-02	Admin	maintain the rate card in one protected place	all pricing is consistent and locked
DAB-03	User	use a clear, role-aware navigation shell	I can move between sections easily

Key acceptance criteria — DAB-01: Firebase Auth email/password; role as custom claim; a Sales user cannot read another rep's campaign doc (verified in rules). DAB-02: `rateCard` seeded from KBC Digital Rate Card 2024/25; write = Admin only; version/updatedAt on every edit. DAB-03: role-aware nav; KBC brand styling; passes a 375px mobile viewport test.

Epic B — Campaign Configuration (Sales)

ID	As a...	I want to...	So that...
DAB-10	Sales executive	capture client & campaign basics	the Order Sheet header is correct
DAB-11	Sales executive	add digital products with their specific details (platforms, posts/day, stream times, flight dates)	scope is fully defined, never vague
DAB-12	Sales executive	see the running total update live as I build	I know the value before I finish

Key acceptance criteria — DAB-11 is the anti-ujanja core: no product can be added without its mandatory quantity fields; the system computes totals (e.g. platforms × posts/day × days). DAB-12: subtotal, VAT 16%, grand total all live; **no editable price field anywhere in the Sales UI** — every price comes from `rateCard`.

Epic C — Discount & Approval

ID	As a...	I want to...	So that...
DAB-20	Sales executive	request a discount on the record	I can offer a concession transparently
DAB-21	Advertising Manager	approve or reject discount requests	concessions are controlled and visible

Key acceptance criteria — DAB-20: system shows rate-card price vs discounted price vs concession value; an Order Sheet cannot be generated while a discount is pending. DAB-21: Ad Manager queue with full context; approve/reject writes approver + timestamp to `auditLog`.

Epic D — Order Sheet Generation (the centrepiece)

ID	As a...	I want to...	So that...
DAB-30	System	mint a unique DAB reference on generation	the Order Sheet links to the paper order
DAB-31	Sales executive	click Print/Download and get a finished, locked Order Sheet	the client can sign it
DAB-32	Business	see boosting shown as an explicit line	it is never a hidden footnote

Key acceptance criteria — DAB-30: format `DAB-YYYY-NNNNN`, sequential, unique, never reused. DAB-31: PDF generated **server-side** (Cloud Function), KBC letterhead, all sections, flattened/non-editable, stored to Firebase Storage; if a discount is approved, the **Approved Discount block** prints (rate-card price, discounted price, %, Ad Manager signature line). DAB-32: prints platforms, budget, and whether inside or on top of the order value; "NIL" if none.

Epic E — Signature, Countersign & Payment Gate

ID	As a...	I want to...	So that...
DAB-40	Sales executive	mark the client signed & enter the Air-Time serial	the campaign can progress
DAB-41	Advertising Manager	countersign the Order Sheet	internal approval is on record
DAB-42	Finance / Ad Manager	confirm deposit or LPO	work never starts unpaid

Key acceptance criteria — DAB-40: upload signed scan; Air-Time serial required. DAB-42: **hard gate** — Digital Ops brief stays locked until payment confirmed; no Sales override.

Epic F — Digital Execution

ID	As a...	I want to...	So that...
DAB-50	Digital Ops	see only unlocked, fully-approved briefs	I execute clean orders
DAB-51	Digital Ops	track and validate client materials against spec	nothing goes live off-spec
DAB-52	Digital Ops	log each element going live with a timestamp	we have an auditable delivery record
DAB-53	Digital Ops	attach proof of delivery per campaign	delivery is evidenced
DAB-54	Digital Ops	generate a report pre-filled from the campaign	I only add delivered actuals

Epic G — Change Orders

ID	As a...	I want to...	So that...
DAB-60	Sales executive	raise a Change Order (DAB-CO) for added scope	extra work is documented and billed, not absorbed

Key acceptance criteria — references parent campaign; adds rate-card-priced line items; mints `DAB-CO-YYYY-NNNNN` ; runs the same approve → countersign → payment chain; Digital Ops sees it as a distinct additive instruction and never acts on verbal additions.

Epic H — Management Visibility

ID	As a...	I want to...	So that...
DAB-70	Management	see real-time pipeline & booked revenue	digital revenue is finally measurable
DAB-71	Management	see all discounts given (rep, value, %, approver)	concessions are governed

6. Version 1 — Scope Definition

6.1 In Scope (Must Ship)

- Firebase Auth login; four roles enforced in security rules (Sales, Ad Manager, Digital Ops, Management/Admin)
- Locked rate-card engine — all prices server-set from the rate card; no editable price fields for Sales
- Campaign configurator with per-product detail capture and a live cost calculator (subtotal, VAT 16%, total)
- Discount request + Advertising Manager approval workflow
- **Server-side generation of the locked, KBC-branded Order Sheet PDF** with auto DAB reference
- Explicit boosting block and Approved Discount block on the Order Sheet
- Signed-sheet upload + Air-Time serial capture; Ad Manager countersign; payment-confirmation hard gate
- Digital Ops execution dashboard: material validation, go-live logging, POD upload, report generation
- Change Order (DAB-CO) workflow
- Management dashboard: pipeline, booked revenue, overdue flags, discount report
- Immutable audit log on every state change
- Mobile-responsive (Android + iOS browsers); deployed on Vercel

6.2 Out of Scope for V1 (Backlog — see Section 8)

- Auto invoicing & finance-system integration
- Client-facing self-service portal
- Google Ad Manager / programmatic integration
- Native iOS/Android app
- TV and Radio modules
- Automated posting to social platforms via APIs
- E-signature on the Order Sheet (V1 uses print + wet signature + scan upload)

RULE FOR THE TEAM

If a stakeholder asks for a backlog item during V1, write it in the backlog and say "logged for V2." Do not build it. Scope creep is what kills internal tools before they ship.

7. Sprint Plan — Phased, 2-Week Sprints

Each sprint ends with something that **works and is demoable**, not a mockup. Demo to the Product Owner at the end of every sprint. We ship in **two phases**: Phase 1 delivers the Order Sheet slice — the piece that stops the bleeding — as fast as possible; Phase 2 adds execution, reporting and management visibility.

Phase 1 — The Order Sheet slice (ship this first)

Sprint	Deliverables	Goal
Sprint 0 (Days 1–2)	Repo, Firebase project, Vercel pipeline, auth scaffold, agreed folder structure, Firestore schema, rate-card seeded	Working skeleton deployed — you can log in
Sprint 1	DAB-01, DAB-02, DAB-03 — auth, roles in rules, rate-card engine, app shell	Login + roles work; rate card live & Admin-editable
Sprint 2	DAB-10, DAB-11, DAB-12 — client form, product configurator, live calculator	A Sales user builds a priced campaign on a phone
Sprint 3	DAB-20, DAB-21 — discount request & Ad Manager approval	Discounts are controlled and on-record
Sprint 4	DAB-30, DAB-31, DAB-32 — DAB ref, locked Order Sheet PDF , boosting & discount blocks	PHASE 1 SHIP A signed-ready Order Sheet can be generated

PHASE 1 MILESTONE

At the end of Sprint 4 the team has shipped the single most valuable capability: a salesperson can configure a campaign and generate a locked, itemised, priced Order Sheet for the client to sign. The ambiguity problem is solved at this point, even before execution tooling exists. Put this in front of real sales execs and gather feedback before Phase 2.

Phase 2 — Execution, reporting & oversight

Sprint	Deliverables	Goal
Sprint 5	DAB-40, DAB-41, DAB-42 — signed-sheet upload, Air-Time serial, countersign, payment gate	The hard gate works; nothing executes unpaid
Sprint 6	DAB-50, DAB-51, DAB-52 — execution dashboard, material validation, go-live logging	Digital Ops works only clean, unlocked briefs
Sprint 7	DAB-53, DAB-54 — POD upload, pre-filled report generation	Delivery is evidenced; reports are one click
Sprint 8	DAB-60 — Change Order (DAB-CO) workflow	Scope growth is documented and billable
Sprint 9	DAB-70, DAB-71 — management dashboard + discount report	Live pipeline, revenue & discount visibility
Sprint 10	UAT with 2 sales execs + 2 digital staff, security-rule testing, audit-log verification, training guide	FULL V1 SHIP

Dependencies: Sprint 2 depends on Sprint 1; Sprint 4 depends on Sprint 2; Sprint 5 depends on Sprint 4. Respect these. Sprints may overlap if the team grows, but not across a dependency.

DAILY CHECK-IN RULE

Each team member posts a one-line update every morning: what I shipped yesterday, what I am building today, any blocker. This replaces lengthy standups — same rule as the Lineup build.

8. Product Backlog — V2 & Beyond

Confirmed for future versions. Priorities set with Sales, Digital, and Management after V1 is live.

Feature	Version	Priority	Notes
Auto invoice generation from delivery records	V1.5	High	V1 exports data for manual finance entry records
Direct integration with KBC finance system	V1.5	High	Needs finance-system API & sign-off
Email / push notifications for approvals & SLAs	V2	High	Makes the workflow active, not passive
Client-facing self-service portal	V2	Medium	V1 keeps clients behind the sales exec
E-signature on the Order Sheet	V2	Medium	Removes print + scan step
Google Ad Manager / programmatic integration	V2	Medium	Large scope; not needed to fix core problem
Automated posting to social platforms via APIs	V2	Low	Execution stays manual in V1
TV AdBoard module	V2.0	Medium	Separate product version — same model
Radio AdBoard module	V2.5	Medium	Station-by-station rate cards
Native iOS/Android app	V3	Low	V1 responsive web is sufficient
Unified cross-platform booking (TV+Radio+Digital)	V3	Low	The long-term KBC AdBoard vision

9. Technical Decisions

Layer	Choice	Rationale
Frontend	React JS (Vite)	Same as Lineup; dynamic UI; interns know it; functional components + hooks only
Backend / DB	Firebase (Firestore + Auth)	Managed auth, real-time DB, no server to maintain, free tier covers V1
PDF generation	Server-side Cloud Function	Order Sheet must be tamper-proof — never generate it client-side
File storage	Firebase Storage	Creative uploads + generated PDFs
Hosting	Vercel	Git-integrated deploys; instant rollbacks; matches Lineup
Mobile	Responsive web (no native app)	No app-store friction; works on any device

9.1 Firestore collections (design in Sprint 0)

- `users` — uid, name, email, role, phone, active
- `rateCard` — productId, category, name, platform, unit, unitPrice, version (Admin-write only)
- `campaigns` — the central record: dabRef, airtimeOrderSerial, status, client{}, campaign{}, lineItems[], boosting{}, discount{}, totals{}, payment{}, orderSheetPdfUrl, signatures{}
- `changeOrders` — cold, dabCoRef, parentCampaignId, additionalLineItems[], status, signatures{}
- `materials` — materialId, campaignId, type, fileUrl, specStatus, dueDate
- `executionTasks` — taskId, campaignId, productId, taskName, slaDeadline, status, goLiveTimestamp
- `auditLog` — append-only; logId, campaignId, action, actor, role, timestamp, detail

Status enum (drives the whole UI): `draft` → `discountPending` → `discountApproved` → `orderSheetGenerated` → `clientSigned` → `adManagerCountersigned` → `paymentConfirmed` → `briefUnlocked` → `inExecution` → `delivered` → `reported` → `closed`

9.2 Security rules — get these right early

The entire anti-ujanja value collapses if rules are weak. Write rule tests; do not trust the UI to enforce anything that matters.

- `rateCard`: read = any authed user; **write = Admin only**
- `campaigns`: Sales reads/writes only own docs; Digital Ops reads docs at status ≥ `briefUnlocked`; Ad Manager writes only discount/countersign fields
- Line-item prices must be **server-set** from `rateCard` — reject any client-supplied price
- `discount.status = "approved"` transition allowed **only** for the Ad Manager role
- `auditLog`: create = allowed; **update/delete = nobody** (immutable)

10. Definition of Done

Per story — done only when ALL are true:

1. Works on a mobile viewport (test at 375px wide first)
2. Role access verified in Firestore security rules, not just hidden in the UI

3. Writes an `auditLog` entry wherever it changes campaign state
4. No editable price field exposed to Sales anywhere
5. Demoed to the Product Owner and accepted
6. Code-reviewed by at least one other team member
7. No console errors; loading and empty states handled

V1 overall — ready to ship when: all stories implemented and testable; deployed on a public Vercel URL; login works for all four roles; a sales exec can configure a campaign and generate a correct locked Order Sheet in under 10 minutes on a phone; the payment gate provably blocks execution; the audit log is provably immutable; the Product Owner has signed off the live build.

11. Brand Tokens (use these — don't invent colours)

Token	Value	Use
Navy	<code>#1A3E6F</code>	Primary, headers, section bars
Gold	<code>#C8972B</code>	Accent, DAB reference highlight, key totals
Teal	<code>#0F6E56</code>	Role/ownership tags, success states
Red	<code>#B71C1C</code>	Warnings, the "don't start without 3 things" notice
Off-white	<code>#F7F7F7</code>	Alternating table rows

Font: clean sans-serif (Arial / Inter). Generous spacing — remember the 50-something Sales persona. This is not a dense trading terminal.

12. What Success Looks Like

After V1 ships, success is not measured by features — it is measured by behaviour change. 90 days after launch:

- 100% of digital campaigns have a signed, itemised Order Sheet
- 100% of discounts are documented and Advertising-Manager approved
- Zero undocumented scope additions — all growth goes through Change Orders
- Payment is confirmed before execution on 100% of campaigns
- Management can see live pipeline and total discounts given — for the first time ever

If the app does those things reliably, it has succeeded — regardless of how many "nice to have" features it lacks. This is a learning project, but it is also a real tool that real people will use to protect real revenue. **Ship the spine. Keep it simple. Make it better in V2.**